HOL2: Backend Reliability Challenge - Handle It Smart

Deliverables: middleware script, retry policy explanation, performance reflection

1. Middleware Script

middleware.py (class `LLMMiddleware`) wraps chat.completions with: **retry/backoff** (max 3 retries, delay = min(1s x 2^attempt, 16s) ± 200 ms jitter), **timeout handling** (30 s client timeout, APITimeoutError is retried), **caching** (in-memory, MD5 prompt hash, hit returns instantly with cache_hit=true) and **structured JSONL logging** to logs/run_*.jsonl (timestamp, prompt_hash, latency_ms, retry_count, input/output tokens, cost_estimate, cache_hit, success, error). Retryable: 429, 5xx, timeouts, connection errors. Fail fast: 4xx.

Evidence (simulated provider). The real middleware was run against a fake client that injects failures, because no live API key was available when this report was generated. Backoff waits and logged retry_count are genuinely measured; the 150 ms model time and token counts are synthetic. Re-run `python middleware.py` with a key for live numbers.

Scenario	API calls	Retries	Latency (ms)	Result	Cache
S1 clean call	1	0	150.2	success	-
S2 one 429 then success	2	1	1140.7	success	-
S3 timeout then success	2	1	1090.2	success	-
S4 three 500s then success	4	3	7144.1	success	-
S5 429 forever (exhausted)	4	3	7041.5	fail: Rate limit exceeded	-
S6 401 (fail fast)	1	0	0.1	fail: Invalid API key	-
S7 repeat of S1 (cache hit)	0	0	0.0	success	hit

2. Retry Policy Explanation (149 words)

The middleware retries up to three times, so a request makes at most four attempts. One retry fixes most momentary 429s and timeouts, while three covers a short outage without a wait beyond about seven seconds. Delays double on each retry (1s, 2s, 4s, capped at 16s) so the provider gets progressively more time to recover. Random jitter of plus or minus 200 ms is added so that many clients who failed together do not all retry at the same instant (the thundering herd). Retries stop after the third retry, on the first success, or immediately for errors that will never succeed: 400, 401, 403, 404 and 422 fail fast with no sleep. Only 429, 5xx, timeouts and connection errors are retried. Bounded attempts and jitter keep our traffic from piling onto an already overloaded API, which is what prevents a local blip from cascading into a wider outage.

3. Performance Reflection (187 words)

Retries help when failures are transient: in the simulation a single 429 cost about one extra second but turned an error into an answer. They hurt when the failure is persistent, because an exhausted request burned about seven seconds of backoff and still failed, and retried traffic adds load to a provider that is already throttling us. Non-retryable errors such as 401 should fail fast, and the simulation returned one in under a millisecond. Caching changes the dynamics for deterministic, repeated prompts: the repeated prompt was served in effectively zero time with zero cost and no API call, though it is only safe at temperature 0 and can return stale answers. p95 matters more than p50 because retries create a long tail: the median request may be fast while one in twenty waits through several backoffs, and users judge a service by its worst experiences, while

timeouts and capacity are sized to the tail. Middleware design therefore decides real user experience: bounded retries, jitter, caching and per-request logs of retry_count and latency trade a little median latency for far fewer visible failures, and make that trade measurable.